

Dodo Alive! Team 6: Tron1 Locomotion

You-Ri Su^{1,*}, Benjamin Knobel del Olmo¹, Vaidehi Bagria¹, Salma Abass¹, Ruicong Zhang¹

Abstract—We aimed to train a locomotion policy for Tron1 using reinforcement learning. Our work started by reproducing the developer’s policy and progressed through Isaac Gym training, transfer to MuJoCo, and deployment in Isaac Lab. The project involved reward design, training on multiple hardware configurations, and a successful sim-to-sim transfer. Furthermore, collaboration with RoboTUM demonstrated that our workflow was reusable by other groups. This report is intended to be self-sufficient so that future work can continue seamlessly from what we describe here.

I. INTRODUCTION

This document serves two purposes: Firstly, it showcases the work done in the SS 2025 by the above mentioned authors. Secondly, it is meant a strong starting point and overview for the next generation of Dodo Alive! participants to continue to iterate on our findings. As we were the first team to work with the Tron1 Robot by LimX Dynamics, we have mainly aimed this semester to lay the groundwork and explore multiple avenues of developing, to be able to give guidance about the choices in developing hardware and software.

Our task was to implement a learning based locomotion on the Tron1, which is part-owned by MIRMI and by RoboTUM (a student club). Learning-based locomotion has emerged as a powerful approach for legged robots, leveraging reinforcement learning (RL) to discover robust walking strategies that are difficult to hand-design. The primary objective was to train a policy capable of velocity tracking and stable walking across multiple environments.

We began by reproducing the developer’s reference policy to build confidence in the framework and verify our environment setup. In parallel, the team studied key learning materials such as the Deep RL Bootcamp [1] (Chapter 1-4), HuggingFace RL course [2], and implementing a small RL project (a AI that plays Snake) by following a YouTube tutorial by Patrick Loeber [3]. These steps are very important to get a solid understanding of the Algorithms you will be working with during this semester. In our opinion, the Deep RL Bootcamp provided the best inside into the mathematics, while the SnakeAI project gave the best hands-on understanding.

We then went on to study the documentation of the developers of the Tron1 robot [4], where they explain how to recreate their policy. As you will probably have access to some form of current hardware, you will probably not

be able to recreate their exact steps in native Ubuntu 20 (Linux), the same problem we faced (more on Hardware and Compatibility issues see Hardware Setup Section). This is why the developers provide a Docker image, which is basically a full-fledged instruction set for a virtual machine on which operating system to simulate and which dependencies to download. Be advised that the image is quite large (30 GB).

The developers used Isaac Gym to train their RL policy, which has since been discontinued by NVidia. We spend roughly half the semester working with the Isaac Gym environment of the developers and the other half in the effort to transfer everything over to IsaacSim, the newest simulation environment of NVidia which also benefits from a very powerful Learning Framework called IsaacLab. IsaacLab makes it much easier to implement a large variety of Learning algorithms and has a cleaner structure in general, which supports development. The following chapters will be about our work using both, first in our mission to recreate the developers’ policy and secondly in our goal to transfer to IsaacSim / Lab to enable further native development.

II. BACKGROUND AND METHODOLOGY

A. Tron1

The experiments in this work were conducted on the **Tron1** robot platform. Tron1 is a multimodal bipedal robot that supports multiple foot attachments, including point-foot, sole-foot, and wheeled configurations [5]. In our experiments, we used the point-foot variant. The robot’s kinematics are described in URDF format, which defines the links, joints, and inertial properties required for accurate simulation [6]. Furthermore, Tron1 provides a fully open SDK and hardware interface, allowing seamless integration between simulation and real-world deployment.

B. Reinforcement learning techniques

To train such robots, usually Reinforcement Learning techniques (RL) are used. RL is a control paradigm where an agent learns which action to take based on its interaction with its environment. It is particularly well-suited for locomotion because it can handle continuous, high-dimensional action spaces and adapt to nonlinear dynamics that are difficult to control with classical methods. The most widely adopted algorithms for robotic control include *Proximal Policy Optimization (PPO)* [1], which strikes a balance between stability and efficiency, *(Soft) Actor-Critic (SAC)* [2], which leverages entropy maximization for exploration, and *Deep Deterministic Policy Gradient (DDPG)* [1], which extends RL to continuous deterministic policies. These approaches

¹The authors are with the Chair of Robotics and Systems Intelligence and the Munich Institute of Robotics and Machine Intelligence (MIRMI), Technical University of Munich, Germany.

The work has been done as a part of DODO Alive! WS2025

generally follow the **actor–critic** framework, where the actor outputs actions and the critic evaluates them via a learned value function [2].

C. Methodology

The Tron1 model was configured both in Isaac Gym and Isaac Lab environments. Isaac Gym enabled massively parallel training of thousands of environments in headless mode, while Isaac Lab, which is based on NVIDIA PhysX, provided a research-oriented framework on top of Isaac Sim [7].

A crucial element of RL training for locomotion is the **reward function design**. The reward function must encourage desirable behaviors—such as upright posture, forward velocity tracking, and smooth turning—while penalizing undesirable outcomes like falling, excessive joint torques, or unstable oscillations. Balancing these rewards and penalties is essential for convergence, as it directly shapes the learned locomotion policy. This will be further discussed below.

III. TRAINING AND RESULTS

A. Isaac Gym

Training in Isaac Gym allowed us to quickly achieve stable walking behaviors thanks to its massively parallel rollout capability. By leveraging thousands of parallel environments on the GPU, we were able to dramatically speed up data collection and policy learning compared to CPU-based simulators.

Policies trained in Isaac Gym generalized well and were successfully transferred to MuJoCo, where the robot demonstrated stable, repeatable walking behavior, validating strong sim-to-sim generalization.

a) Reward Design.: The reward function in Isaac Gym was already designed to encourage stable locomotion. The main components included:

- **Forward Velocity Reward**: Proportional to the robot’s COM velocity along the target axis, encouraging faster progress.
- **Energy Efficiency Penalty**: Penalizing excessive torque and joint accelerations to promote energy-efficient gaits.
- **Balance Maintenance**: Penalizing large base orientation deviations to maintain upright posture.
- **Smoothness Term**: Rewarding low jerk in joint trajectories for more natural motion.

After a bit of tuning from our side, this reward shaping led to fast convergence and reduced policy instability.

b) Experiment Tracking with W&B: We integrated Weights & Biases (W&B) into our Isaac Gym codebase to enable real-time logging of rewards, losses, and episode statistics. This allowed us to:

- Track convergence and training stability across different hyperparameter settings.
- Visualize learning curves and identify runs with better sample efficiency.
- Automatically compare experiments, making it easier to tune reward weights and training schedules.

The corresponding code modification can be found in our GitHub repository (Commit link: [8], Repository: [9]).

c) Custom Docker + GPU Setup: To ensure reproducibility and scalability, the developers built a custom Docker image with:

- CUDA-enabled PyTorch and Isaac Gym dependencies pre-installed.
- X11 display access for visualizing rollouts in real-time from within the container.
- Export scripts to package trained policies and robot configurations for deployment.

This containerized setup allowed us to quickly spin up new training instances on different machines, enabling parallel experimentation. This will be further explained in the following section

d) Tron1 Training.: The Tron1 robot was trained entirely inside the containerized Isaac Gym environment. The pipeline supported:

- **Batch Parallelism**: Thousands of Tron instances simulated simultaneously on the GPU.
- **Curriculum Learning**: Starting from simpler walking tasks (flat terrain) and progressively introducing perturbations and uneven surfaces.
- **Export for Deployment**: Trained policies were exported to ONNX format and deployed in MuJoCo for validation.

This approach yielded robust policies that performed well under dynamics mismatch between Isaac Gym and MuJoCo.

Overall, this combination of a reproducible containerized workflow, efficient GPU-based simulation, and systematic reward design enabled us to train robust walking policies significantly faster than with traditional CPU-based simulators.

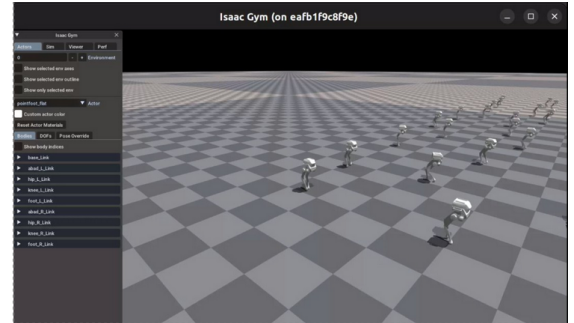


Fig. 1. Training process in Isaac Gym showing parallelized Tron LIMX environments running on GPU.

B. Isaac Sim & Isaac Lab

1) Introduction: Isaac Sim and Isaac Lab are two tools in NVIDIA’s robotics development platform. Isaac Sim is built on NVIDIA Omniverse as a full-stack robotics simulation tool, designed to provide developers with a physically accurate and visually realistic virtual environment, supporting a complete workflow from perception training to control validation. Isaac Lab is an

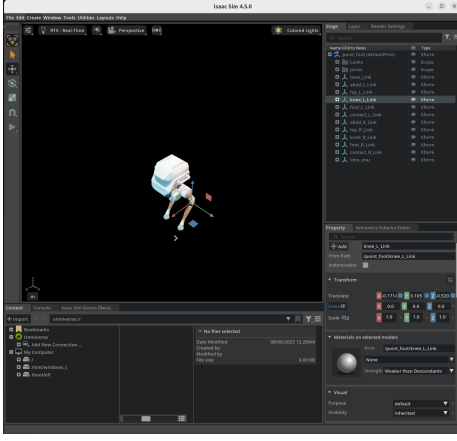


Fig. 2. Bipedal Robot Tron1 in Isaac Lab.

```

23 feet_air_time = RowTerm()
24 func=mdp.feet_air_time_positive_biped,
25 weight=0.25,
26 parameters=[
27     "command name": "base_velocity",
28     "sensor_cfg": SceneEntityCfg("contact_forces", body_names="contact_*_Link"),
29     "threshold": 0.4,
30 ],
31 ),
32 ),
33 feet_slide = RowTerm()
34 func=mdp.feet_slide,
35 weight=0.1,
36 parameters=[
37     "sensor_cfg": SceneEntityCfg("contact_forces", body_names="contact_*_Link"),
38     "asset_cfg": SceneEntityCfg("robot", body_names="contact_*_Link"),
39 ],
40 ),
41 ),
42 # Penalize deviation from default of the joints that are not essential for locomotion
43 joint_deviation_hip = RowTerm()
44 func=mdp.joint_deviation_hip,
45 weight=0.1,
46 parameters=[
47     "asset_cfg": SceneEntityCfg("robot", joint_names=["hip.*"]),
48 ],
49 ),
50 joint_deviation_ll = RowTerm()
51 func=mdp.joint_deviation_ll,
52 weight=0.1,
53 parameters=[
54     "asset_cfg": SceneEntityCfg(
55         "robot",
56         joint_names=[
57             "base.*",
58             "hip.*",
59             "knee.*",
60 ],
61 ),
62 ],
63 ),
64 ),
65 ),
66 ),
67 ),
68 ),
69 ),
70 ),
71 ),
72 ),
73 ),
74 ),
75 ),
76 ),
77 ),
78 ),
79 ),
80 ),
81 ),
82 ),
83 ),
84 ),
85 ),
86 ),
87 ),
88 ),
89 ),
90 ),
91 ),
92 ),
93 ),
94 ),
95 ),
96 ),
97 ),
98 ),
99 ),
100 ),

```

Fig. 3. Alignment of Reward Terms with Body Names of Tron1

open-source robotics learning framework based on Isaac Sim, specifically designed for Reinforcement Learning (RL) and Imitation Learning (IL), providing a full-process tool chain from environment setup to policy deployment.

2) *Environment Setup*: We first locally installed Isaac Sim and Isaac Lab on the Ubuntu 22.04 operating system according to the *Isaac Lab Documentation* [10]. Subsequently, we utilized the tool provided by Isaac Lab to convert the official URDF file of Tron1 [6] into a USD file, which was then successfully imported into Isaac Lab. The figure 2 shows the bipedal robot Tron1 in Isaac Lab.

3) *Prepare for Training*: We had a meeting with Shafeef, who provided us with valuable insights into Isaac Lab [11]. Following his tutorial, we made modifications to the code for the bipedal robot Unitree G1 within Isaac Lab. A crucial aspect of this process was ensuring that the reward terms were correctly matched with the corresponding body names of Tron1, which can be found in the URDF. The figure 3 illustrates three examples of how reward terms are matched with body names.

4) *Learning Curves*: As previously discussed, we utilized Wandb to visualize our training process. Among the various metrics, three key curves stand out as particularly important.

The first is a reward curve named *track_lin_vel_xy_exp*, which incentivizes Tron1 to closely track the target linear velocity within the XY plane. This control target is directly linked to the robot's forward speed. The second curve, *track_ang_vel_z_exp*, is also a reward curve. It encourages the robot to track the target's angular velocity (specifically, rotation around the Z-axis), a critical factor for optimizing the robot's steering control. The third and final curve is a punishment curve named *flat_orientation_l2*. This curve penalizes deviations of the robot's torso from the horizontal plane, which is essential for maintaining balance. The figures 4, 5 and 6 illustrate the convergence of *track_lin_vel_xy_exp*, *track_ang_vel_z_exp*, and *flat_orientation_l2* after 15k iterations of training.

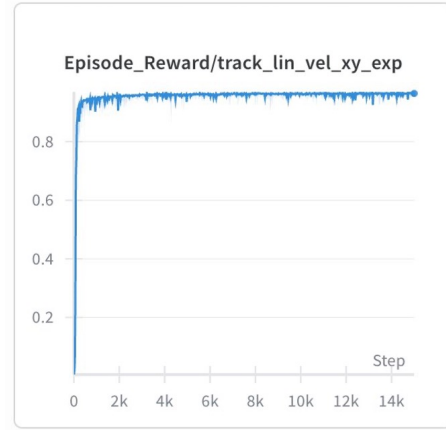


Fig. 4. *track_lin_vel_xy_exp*

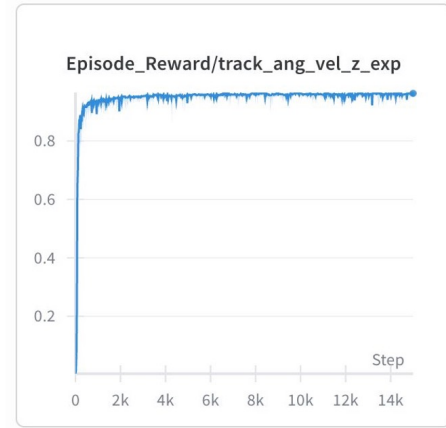


Fig. 5. *track_ang_vel_z_exp*

C. Video Results

The trained robot demonstrated locomotion in flat environments as well as on stairs and uneven ground. Parameter tuning enabled walking with higher robustness, and the robot successfully performed turning maneuvers. Videos qualitatively confirmed stability and gait diversity. Related videos are available within the Isaac Lab section of Notion [12],

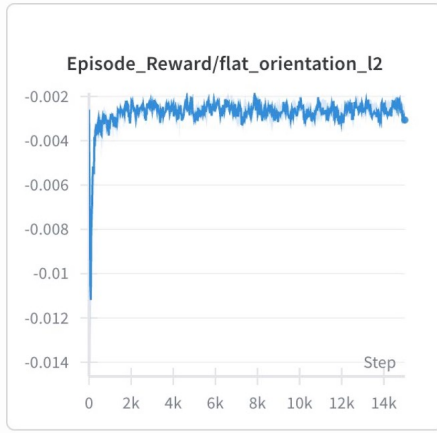


Fig. 6. *flat_orientation.l2*

which are named as "Locomotion in Flat Environments" and "Locomotion on Stairs and Uneven Ground".

IV. HARDWARE SETUP

Training and visualizing reinforcement learning policies in Isaac **Gym** requires specific hardware and software conditions: Ubuntu 20.04 and a GPU with at least an NVIDIA RTX 3070 (ideally RTX 4080). Since these requirements were not available on all members' laptops, we explored multiple deployment paths. Our setup was initially based on the tutorial provided by limx [4], which describes how to configure a Docker environment and pull their pre-built image. We recommend that readers consult this tutorial to familiarize themselves with the general workflow. However, to serve our specific needs, we **adapted** several of the commands and configurations, as detailed in the following. In this section, we document the exact steps of our Docker-based setup and summarize the pros and cons of each approach.

A. Docker-Based Setup

We first cloned the project repository onto the host system. To set up Docker and store log data from the container into the host file system, we used:

```
sudo docker run -it --gpus all --rm \
-e ROBOT_TYPE=PF_TRON1A \
-v~/limx_rl_logs:/workspace/limx_rl
/pointfoot-legged-gym/logs\
crpi...aliyuncs.com/limx-rl/base:v10
```

where the full image name is

crpi-yznuf1hnz012tr44.cn-shanghai.personal.cr.aliyuncs.com/limx-rl/base:v10.

The log files are now persisted under /limx_rl_logs on the host.

Then to start training, we ran:

```
cd limx_rl/pointfoot-legged-gym
python legged_gym/scripts/train.py --task=pointfoot_flat \
--headless --num_envs 1024
```

Parameters:

- `--task=pointfoot_flat`: specifies the environment type.
- `--headless`: runs without a GUI (faster).
- `--num_envs 1024`: spawns 1024 parallel environments for faster training.

To resume training from a checkpoint, we ran:

```
python legged_gym/scripts/train.py --task=pointfoot_flat \
--resume --headless --load_run Apr16_16-51-06_
--checkpoint 200
```

where checkpoint corresponds to the suffix of the stored model file (e.g., model_200.pt)

1) *Code Debugging via Bind Mounts*: To reflect code changes made on the host system directly inside the container, we mounted the project directory as a bind volume:

```
sudo docker run -it --gpus all \
-v /home/user/limx_rl/pointfoot-legged-gym:\
/workspace/limx_rl/pointfoot-legged-gym \
crpi...aliyuncs.com/limx-rl/base:v10
```

2) *Rendering Support*: To enable GUI rendering inside Docker, we first gave X11 permissions:

```
xhost +local:root
```

and then launched the container with the display forwarded:

```
sudo docker run -it --gpus all \
--env="DISPLAY=$DISPLAY" \
--env="NVIDIA_DRIVER_CAPABILITIES=all" \
--env="XAUTHORITY=$XAUTHORITY" \
-v /tmp/.X11-unix:/tmp/.X11-unix \
-v /home/user/limx_rl/pointfoot-legged-gym:\
/workspace/limx_rl/pointfoot-legged-gym \
crpi...aliyuncs.com/limx-rl/base:v10
```

3) *Persisting Custom Environments*: To avoid reconfiguration after every container restart, we committed the container into a custom image:

```
sudo docker commit <container_id> my_custom_rl_env:v1
```

This allowed us to later restart training with:

```
sudo docker start -ai limx_container_v2
```

This setup ensured that dependencies (CUDA, Isaac Gym, PyTorch, etc.) were already pre-installed, avoiding common compatibility issues when configuring the environment on different systems.

B. Remote Laboratory PC via AnyDesk

We also had access to a shared PC, running Ubuntu 22.04 and equipped with a suitable GPU. Remote connection was established using the AnyDesk client. After installing AnyDesk on the local machine, the lab PC was accessed using its desk ID and password [13]. This method allowed us to run experiments with full GPU acceleration, but required manual restarts whenever the PC shut down, limiting its convenience.

C. Google Cloud Virtual Machine

As an alternative to using local hardware, we deployed Isaac Gym on a GPU-enabled Google Cloud virtual machine (VM). This approach allowed us to run both training and visualization without being constrained by the limited GPU capabilities of personal laptops. To set up the environment, we followed several key steps:

- 1) **Google Cloud VM Setup:** We launched a VM with GPU support following the RoboTUM tutorial presented in the Notion page. [14]
- 2) **Docker Environment:** We used a prebuilt Docker image for a desktop-enabled environment, `dorowu/ubuntu-desktop-lxde-vnc`, to facilitate both training and remote GUI access. The container was run with:

```
docker run -d -p 8080:80 --shm-size=512m \
--name gui-desktop dorowu/ubuntu-desktop-lxde-vnc
```

- 3) **Firewall Configuration:** To enable remote access for visualization, we opened port 8080 in the Google Cloud firewall, allowing us to connect through a web browser to the VNC desktop of the container.

While this setup provided significant flexibility and access to high-performance GPUs, it had two main limitations. First, the free Google Cloud credits were consumed quickly during extended training. Second, visualization performance over VNC was limited, making real-time observation less smooth compared to a local GPU setup. Therefore, we recommend using Google Cloud VMs primarily for headless training, exporting the learned policies, and performing visualization locally where GPU resources and display performance are better.

D. Comparison of Deployment Approaches

- **Docker container with GPU** – used by all members, reliable and reproducible baseline.
- **Remote lab PCs (AnyDesk)** – provided access to high-end GPUs, but dependent on lab availability.
- **Google Cloud VM** – flexible and accessible anywhere, but limited by cost and laggy visualization.
- **Local environments (dual boot)** – flexible but computationally limited for large-scale training.

Overall, the most effective setup combined Docker containers for consistency with remote laboratory PCs for reliable GPU access.

V. DISCUSSION AND FUTURE WORK

A central contribution of this work was the successful transfer from Isaac Gym to Isaac Lab, including several trial-and-error phases during setup as we were the first team working on Tron1 robot simulation. This transition is not only technically relevant but also strategically important: future student teams will be able to continue development on an actively maintained framework rather than one that has been discontinued. In addition, the integration of Weights & Biases (W&B) into our Isaac Gym experiments enabled

transparent logging of metrics and facilitated direct comparability with Isaac Lab training runs. This consistency was crucial for validating that the transfer did not only succeed visually but also in terms of quantitative performance. By establishing comparable results in both environments, we created a bridge that ensures continuity of the Dodo Alive! project.

Our semester project further demonstrated the feasibility of sim-to-sim transfer and the advantages of combining Isaac Gym with Isaac Lab. Nevertheless, several limitations remain:

- More diverse training cases are required for robustness.
- Deployment of trained policies on a real Tron1 robot is an open task.
- Reward shaping and curriculum learning can be further optimized.

VI. CONCLUSION

We successfully trained locomotion policies for Tron1 using reinforcement learning, starting from Isaac Gym, validating in MuJoCo, and finally extending to Isaac Lab. The project not only deepened our understanding of RL and simulation platforms but also highlighted the importance of workflow design, cross-simulator evaluation, and collaboration. By integrating Weights & Biases (W&B) into our Isaac Gym training pipeline, we ensured consistent logging and enabled direct comparison with Isaac Lab results. This made the experiments reproducible and transparent. A detailed record of intermediate steps, including experimental trials and failed attempts, can be found in our project Notion log [15]. Future work will focus on deploying policies on the real Tron1 robot, expanding environment diversity, and improving reward shaping strategies.

REFERENCES

- [1] “Deep RL Bootcamp.” <https://sites.google.com/view/deep-rl-bootcamp/lectures>. Accessed: Sept. 2025.
- [2] “Hugging Face Deep Reinforcement Learning Course.” <https://huggingface.co/learn/deep-rl-course/unit0/introduction>. Accessed: Sept. 2025.
- [3] “Teach AI To Play Snake - Reinforcement Learning.” <https://www.youtube.com/watch?v=PJl4iabBEz0>. Accessed: Sept. 2025.
- [4] “Tron1 Documentation by LimX Dynamics.” <https://support.limxdynamics.com/en/docs-center>. Accessed: Sept. 2025.
- [5] “LimX Dynamics website.” <https://www.limxdynamics.com/en/tron1>. Accessed: Sept. 2025.
- [6] LimX Dynamics, “Tron1 URDF File.” https://github.com/limxdynamics/tron1-rl-isaacgym/blob/master/resources/robots/PF_TRON1A/urdf/robot.urdf. Accessed: Sept. 2025.
- [7] S. Omar and contributors, “Isaac Lab: An Open-Source Robotics Learning Framework.” <https://github.com/shafeef901/IsaacLab>. Accessed: Sept. 2025.
- [8] Y. Su, “GitHub Commit: Weights & Biases Integration for Isaac Gym.” <https://github.com/suyourice/IsaacGym/commit/0dd87c0458e12aa6221dba71f35ea9d7b04c90c6>. Accessed: Sept. 2025.
- [9] Y. Su, “Working Repository of Weights & Biases Integrated Isaac Gym.” <https://github.com/suyourice/IsaacGym>. Accessed: Sept. 2025.
- [10] Isaac Lab Project Developers, “Installation using Isaac Sim pip.” https://isaac-sim.github.io/IsaacLab/main/source/setup/installation/pip_installation.html. Accessed: Sept. 2025.

- [11] Team6, "Shafif tutorial." https://www.notion.so/Isaac-Lab-Isaac-Sim-26e8d877715b80808d29d7b9e3703e58?source=copy_link#2728d877715b80168f68f41ba1628296. Accessed: Sept. 2025.
- [12] Team6, "Isaac Lab." https://www.notion.so/Isaac-Lab-26e8d877715b80808d29d7b9e3703e58?source=copy_link. Accessed: Sept. 2025.
- [13] Team6, "Anydesk connection." [https://www.notion.so/Conn](https://www.notion.so/Connecting-to-PC-Remotely-2718d877715b80dba528c436cd642fc3?source=copy_link)
- [ecting-to-PC-Remotely-2718d877715b80dba528c436cd642fc3?source=copy_link](https://www.notion.so/Connecting-to-PC-Remotely-2718d877715b80dba528c436cd642fc3?source=copy_link). Accessed: Sept. 2025.
- [14] Team6, "VM setting." https://www.notion.so/VM-Setting-2718d877715b80cd863ef203b3f56195?source=copy_link. Accessed: Sept. 2025.
- [15] Team6, "Project Logbook on Notion." [https://www.notion.s](https://www.notion.so/Team-6-Final-Submission-26e8d877715b80708be7de47488ac2ef?source=copy_link)
- [o/Team-6-Final-Submission-26e8d877715b80708be7de47488ac2ef?source=copy_link](https://www.notion.so/Team-6-Final-Submission-26e8d877715b80708be7de47488ac2ef?source=copy_link). Accessed: Sept. 2025.